Android Device Monitor
Desafio técnico prático

Documento de orientação para desenvolvimento, entrega e defesa técnica

ABSTRACT

Desenvolva um aplicativo Android nativo capaz de observar o dispositivo, interpretar sinais de hardware e apresentar evidências em uma interface funcional.

1. 1. Visão geral

O Android Device Monitor é um desafio de desenvolvimento Android para nível estagiário ou residente iniciante. A proposta combina fundamentos de Kotlin, Android SDK, ciclo de vida, interação com sensores, persistência de dados, testes e documentação.

Objetivo central: construir uma janela confiável para o hardware do dispositivo. O aplicativo deve mostrar dados reais, atualizar o acelerômetro em tempo real, registrar leituras e continuar utilizável quando um recurso estiver indisponível.

Contexto curricular: Núcleo Comum de Formação, com carga horária de 210 horas. Duração: cinco dias corridos de desenvolvimento e apresentação técnica de 30 minutos. Pontuação máxima: 100 pontos.

2. 2. Stack e decisões técnicas

Linguagem	Obrigatória: Kotlin.
Plataforma	Obrigatória: Android nativo.
Interface	Jetpack Compose ou XML com Views.
Build	Gradle, com configuração documentada.
Arquitetura	MVVM é recomendada. Outra organização é aceita mediante justificativa técnica.
Persistência	Room é recomendada; SQLite é alternativa aceita.
Testes	JUnit e pelo menos dois testes unitários básicos.
Versionamento	Git com histórico de commits compreensível.

Separação de responsabilidades: a UI apresenta dados e recebe interações; o ViewModel gerencia estado e coordena ações; o Repository centraliza o acesso a sensores e persistência; os Data Sources encapsulam Android SDK e banco.

3. 3. Requisitos funcionais

3.1. RF001 — Informações do dispositivo

Exibir fabricante, modelo, versão do Android e API Level usando dados reais do dispositivo, por exemplo por meio de android.os.Build. As informações devem ser legíveis e não podem estar fixas no código.

3.2. RF002 — Monitoramento de bateria

Apresentar percentual, estado de carregamento e fonte de alimentação quando disponível. Usar BatteryManager e/ou Intent.ACTION_BATTERY_CHANGED. Atualizar o estado quando necessário e informar dados indisponíveis.

3.3. RF003 — Monitoramento de acelerômetro

Capturar eixo X, Y, Z e timestamp usando SensorManager, Sensor e SensorEventListener. A tela deve permitir iniciar e parar o monitoramento, mostrar valores em tempo real e comunicar quando o dispositivo não possuir acelerômetro. Os listeners devem ser liberados no momento adequado do ciclo de vida.

3.4. RF004 — Histórico de leituras

Permitir salvar manualmente a leitura atual, listar registros, excluir registros e manter os dados após fechar ou reiniciar o aplicativo. Modelo mínimo: id Long, timestamp Long, axisX Float, axisY Float e axisZ Float.

3.5. RF005 — Navegação

Disponibilizar telas de informações do dispositivo, monitoramento e histórico. O botão voltar deve funcionar corretamente, sem travamentos ou perda inesperada de estado.

3.6. RF006 — Tratamento de erros

Tratar ausência de acelerômetro, falha ao salvar, histórico vazio e dados de bateria indisponíveis. Mensagens devem ser compreensíveis e a aplicação deve continuar utilizável.

4. 4. Requisitos não funcionais

RNF001 — Desempenho
: A interface permanece responsiva durante leituras e persistência.

RNF002 — Gerenciamento de recursos
: Listeners não ficam ativos desnecessariamente em segundo plano.

RNF003 — Manutenibilidade
: Nomes claros e responsabilidades separadas.

RNF004 — Compatibilidade
: A versão mínima do Android é escolhida e documentada.

RNF005 — Segurança Nenhuma permissão desnecessária ou dado sensível sem justificativa.

RNF006 — Confiabilidade
: Falhas não provocam encerramentos inesperados.

5. 5. Testes mínimos

Implemente no mínimo dois testes unitários. Cenários sugeridos: validar uma leitura de acelerômetro; formatar valores apresentados na interface; tratar valores inválidos; testar uma operação de histórico.

Entregue o código dos testes, o resultado da execução e uma breve descrição dos cenários.

6. 6. Experimentação

O experimento deve demonstrar capacidade básica de coleta e interpretação de dados, sem necessidade de análise estatística avançada.

1. Defina uma frequência ou estratégia de coleta e registre-a.
2. Coloque o dispositivo em repouso e colete uma quantidade definida de amostras.
3. Movimente o dispositivo de forma controlada e colete a mesma ordem de grandeza de amostras.
4. Organize os resultados em tabela ou resumo.
5. Compare os cenários e registre limitações, como orientação do aparelho, ruído e duração da coleta.

Condição A — dispositivo em repouso: espera-se variação menor e componente gravitacional predominante. Condição B — dispositivo em movimento: espera-se maior variação entre eixos e picos de aceleração. Não invente dados: identifique dispositivo, condições, duração e limitações.

7. 7. Entregáveis

ENT001 — Repositório Git
Código-fonte, histórico de commits, Gradle e README.

ENT002 — APK APK de debug instalável.

ENT003 — Documentação
Descrição, requisitos, instruções de execução, arquitetura e limitações.

ENT004 — Testes Dois testes unitários e evidências de execução.

ENT005 — Relatório técnico
Procedimento, resultados, comparação dos cenários e conclusão.

ENT006 — Demonstração
Vídeo de até cinco minutos e apresentação técnica de 30 minutos.

8. 8. Avaliação

Funcionalidades implementadas: 30 pontos. Fundamentos Android e Kotlin: 20 pontos. Arquitetura e qualidade de código: 15 pontos. Sensores e ciclo de vida: 15 pontos. Testes e tratamento de erros: 10 pontos. Documentação e Git: 5 pontos. Apresentação e domínio técnico: 5 pontos.

Faixas: 90–100 Excelente; 75–89 Bom; 60–74 Satisfatório; 0–59 Necessita desenvolvimento.

9. 9. Como fazer: roteiro de implementação

Use a sequência abaixo para reduzir o risco técnico e chegar a uma entrega demonstrável em cinco dias.

1. Defina os estados: captura parada, captura ativa, sensor ausente, histórico vazio e falha de persistência.
2. Crie a base: Kotlin, Gradle, versão mínima documentada e três telas navegáveis.
3. Implemente o dispositivo: modele dados de Build e da bateria antes de conectá-los à UI.
4. Implemente o sensor: registre o acelerômetro, copie os três eixos e o timestamp para um modelo imutável e remova o listener ao pausar.
5. Persista: crie entidade, DAO e banco Room; exponha operações por um Repository.
6. Conecte o estado: use um ViewModel para expor um estado de tela e receber eventos da UI.
7. Teste e observe: escreva dois testes unitários, faça o experimento repouso/movimento e registre as limitações.

Documentação oficial recomendada:
Arquitetura Android: https://developer.android.com/topic/architecture
Sensores: https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview
BatteryManager: https://developer.android.com/reference/android/os/BatteryManager
Room: https://developer.android.com/training/data-storage/room
Testes locais: https://developer.android.com/training/testing/local-tests
Kotlin: https://kotlinlang.org/docs/

10. 10. Desafios opcionais

Inteligência Artificial: exportar leituras em CSV e realizar análise exploratória simples. Segurança Android: analisar permissões e identificar riscos. Eficiência energética: comparar frequências de leitura e impactos no uso de recursos.

11. 11. Critério de conclusão e checklist

O projeto está concluído quando as funcionalidades obrigatórias são demonstráveis, os artefatos são entregues e o candidato explica as decisões técnicas.

- Projeto compila e o APK instala.
- Informações, bateria, sensor e histórico funcionam.
- Persistência sobrevive ao reinício.

- Testes executam.
- README e relatório explicam o caminho.
- Candidato consegue defender as escolhas.